

SVM 教學網站部署

我想要用Manim顯示概念動畫來呈現SVM,然後套用sklearn SVM呈現真實決策面,然後生成一個互動式教學網站部署在streamlit.io上,還有什麼特別需要注意的事項嗎

可以，而且這是一個很適合做成教學作品的題目。不過你要特別注意：**Manim 適合做「概念動畫」**，**sklearn/Plotly 適合做「互動與真實決策面」**，**Streamlit.io 適合做「展示網站」**。不要把所有事情都硬塞進Manim 即時渲染，否則部署會很容易爆。

最重要的注意事項

1. Manim 不建議在 Streamlit 上即時 render

Manim 需要比較多系統依賴，例如 Cairo、Pango、FFmpeg，甚至如果你用 LaTeX 公式動畫，還會需要 TeX 環境。Streamlit Community Cloud 的部署主要靠 `requirements.txt` 管 Python 套件，外部系統套件要用 `packages.txt`，官方也明確區分 Python dependencies 與 external dependencies。👉 [Streamlit 文檔](#)

所以建議你：

把 Manim 動畫預先渲染成 `.mp4` / `.webm`，放在專案的 `assets/videos/` 裡面，Streamlit 用 `st.video()` 播放。

不要每次使用者調整參數就即時呼叫：

```
Bash
manim -pqh svm_scene.py SVMScene
```

這樣在本機可以，但在 Streamlit Cloud 上容易遇到依賴、時間、記憶體或檔案權限問題。

2. 真正互動的部分交給 sklearn + Plotly / Matplotlib

你的互動教學網站可以這樣設計：

使用者調整：

```
C
kernel: linear / rbf / poly
gamma
degree
dataset type
noise
class separation
```

然後用 sklearn 即時訓練：

```
Python
from sklearn.svm import SVC

model = SVC(kernel="rbf", C=C, gamma=gamma)
model.fit(X, y)
```

再畫出：

資料點
decision boundary
margin
support vectors

這一段非常適合用 Plotly 做互動式圖表，或者用 Matplotlib 產生靜態圖。Streamlit 本身就是用來快速建立資料與機器學習互動 app 的 Python 框架。  [Streamlit](#)

3. Streamlit 部署時要準備好依賴檔案

基本專案結構建議：

```
svm-streamlit-teaching/
├── app.py
├── requirements.txt
├── packages.txt
├── assets/
│   ├── videos/
│   │   ├── svm_margin_intro.mp4
│   │   ├── kernel_trick.mp4
│   │   └── support_vectors.mp4
│   └── images/
├── manim_scenes/
│   ├── svm_margin_scene.py
│   ├── kernel_scene.py
│   └── decision_boundary_scene.py
├── utils/
│   ├── svm_model.py
│   ├── plotting.py
│   └── datasets.py
└── README.md
```

requirements.txt 可先放：

```
streamlit
scikit-learn
numpy
pandas
matplotlib
plotly
manim
```

如果你只是播放預先渲染的影片，其實 Streamlit 部署端不一定需要 **manim**。Manim 可以只留在本機開發環境。這樣部署會穩定很多。

如果真的要安裝外部依賴，就需要 **packages.txt**，例如 FFmpeg 之類的系統套件。Streamlit 官方文件指出，外部依賴要透過 **packages.txt** 管理，而不是直接在程式裡跑 **apt-get**。  [Streamlit 文檔](#)

我建議你的教學網站流程

Chapter 1：SVM 概念動畫

用 Manim 呈現：

```
點資料
分類線
margin
support vectors
最大化 margin
```

這裡不要互動，直接做成高品質動畫。

畫面可以像這樣：

```
Step 1：隨機資料點出現
Step 2：出現多條可分割直線
Step 3：比較不同 margin
Step 4：選出最大 margin 的 hyperplane
Step 5：標出 support vectors
```

Chapter 2：數學概念

呈現核心公式：

```
Decision function:
f(x) = w · x + b

Margin:
2 / ||w||

Optimization:
minimize 1/2 ||w||²
subject to y_i(w · x_i + b) ≥ 1
```

這裡可以用 Manim 做公式推導動畫，但 Streamlit 頁面上可以搭配文字說明。

Chapter 3：真實 sklearn SVM 互動

這是網站的主互動區。

左邊 Sidebar：

```
Dataset:
- Linearly separable
- Moons
- Circles
- Blobs

Kernel:
- linear
- rbf
- poly

C:
slider 0.01 ~ 100

Gamma:
slider 0.001 ~ 10

Noise:
slider 0 ~ 0.5
```

右邊顯示：

```
Decision boundary
Support vectors
Training accuracy
Number of support vectors
```

這樣使用者可以直接看到：

```
C 越大：越想分類正確，但可能 overfit
C 越小：允許錯誤，但 margin 較寬
gamma 越大：決策邊界越彎曲，可能 overfit
gamma 越小：決策邊界越平滑
```

Chapter 4：Kernel Trick 視覺化

這一段很重要，因為 SVM 最難懂的地方通常是 kernel。

你可以做兩種呈現：

方式 A：Manim 概念動畫

用 2D 圓形資料點，動畫拉到 3D 空間，讓它變得可分。

方式 B：Streamlit 互動圖

用 `make_circles()` 產生資料，然後讓使用者切換：

```
linear kernel
rbf kernel
poly kernel
```

讓使用者看到 linear 分不開，但 RBF 可以分開。

部署到 streamlit.io 的注意事項

1. Python 版本要固定

Streamlit Community Cloud 會根據支援狀況使用 Python 版本，官方也提醒開發環境和部署環境的 Python 版本要一致，否則可能發生套件相容性問題。🔥 [Streamlit 文檔](#)

建議使用：

```
Python 3.11 或 3.12
```

如果某些套件不支援最新版，就在 Streamlit Cloud 設定裡指定 Python 版本。

2. 不要把大型影片全部塞進 repo

Manim 影片如果很多，GitHub repo 會變很肥。建議：

短影片：可以放 `assets/videos/`
 大型影片：放 YouTube、Google Drive、Cloudflare R2、S3，再嵌入連結

Streamlit 可以用：

```
Python
st.video("assets/videos/svm_margin_intro.mp4")
```

或外部 URL。

3. 避免每次互動都重新訓練太重的模型

SVM 在資料量大時會變慢，尤其是 RBF kernel。

你可以限制資料量：

```
n_samples = 100 ~ 500
```

不要一開始就給 10,000 筆資料。

並且使用快取：

```
Python
@st.cache_data
def generate_dataset(...):
    return X, y
```

或：

```
Python
@st.cache_resource
def train_model(...):
    return model
```

4. 教學網站不要只放模型，要放「觀察提示」

每個互動控制旁邊最好有一句教學提示，例如：

C 越大，模型越不允許錯誤分類，但可能造成 **overfitting**。

Gamma 越大，每個資料點的影響範圍越小，決策邊界會變得更複雜。

這樣才會像「教學網站」，不是只有 demo。

我會建議的最終架構

你的網站可以分成 5 個頁面：

1. SVM 是什麼？
2. 最大 Margin 與 Support Vectors
3. sklearn SVM 互動實驗
4. Kernel Trick 視覺化
5. 小測驗與總結

Streamlit 可以用：

```
Python
st.sidebar.radio(
    "選擇章節",
    [
        "SVM 概念",
        "Margin 與 Support Vectors",
        "sklearn 互動實驗",
        "Kernel Trick",
        "Quiz"
    ]
)
```

最大風險整理

風險	原因	建議
Manim 在 Streamlit Cloud render 失敗	系統依賴多，例如 FFmpeg、Cairo、Pango、LaTeX	預先 render 成影片
部署成功但 app 很慢	SVM RBF 訓練成本較高	限制資料量、使用 cache
影片檔案太大	Manim 影片容易變大	壓縮影片或外部託管
使用者看不懂互動意義	只有 slider 沒有教學提示	每個參數都放說明
本機可以跑，雲端失敗	Python / dependency 版本不同	固定 Python 版本與 requirements
LaTeX 公式 render 出問題	Manim LaTeX 依賴較麻煩	預渲染公式動畫，網站用 Markdown/KaTeX 顯示

結論

你的方向是可行的，但最佳做法是：

Manim：負責高品質概念動畫，預先輸出影片
sklearn：負責真實 SVM 訓練與 decision boundary
Streamlit：負責互動式教學網站與部署
Plotly/Matplotlib：負責即時視覺化

我不建議在 Streamlit Cloud 上即時跑 Manim render。比較穩定的做法是：**Manim 本機產生影片，Streamlit 播放影片；sklearn 和 Plotly 負責互動部分。**

可以 就按照你說的做,可以給我一份明確的prompt讓我丟給claude嗎

可以。下面這份 prompt 你可以直接丟給 Claude / Claude Code / Antigravity Agent, 請它幫你建立完整專案。

你是一位資深 Python、Machine Learning、Manim、Streamlit 教學網站開發者。

我想建立一個「SVM Support Vector Machine 互動式教學網站」, 用來教學 SVM 的核心概念、數學直覺、Support Vectors、Margin、Kernel Trick, 並且使用 sklearn 的 SVM 實際訓練資料, 畫出真實 decision boundary。

請你幫我從零建立完整專案, 並且讓它可以部署到 Streamlit Community Cloud / streamlit.io。

請注意：

不要讓 Streamlit 在雲端即時執行 Manim render。

Manim 只負責製作概念動畫, 先在本機渲染成 mp4 影片。

Streamlit 網站只負責播放這些預先輸出的影片, 並使用 sklearn + Plotly 或 Matplotlib 進行互動式 SVM 視覺化。

一、專案目標

請建立一個完整的 Streamlit 教學網站, 包含以下內容：

1. 用 Manim 製作 SVM 概念動畫
2. 用 sklearn.svm.SVC 訓練真實 SVM 模型
3. 在 Streamlit 裡讓使用者調整 SVM 參數
4. 即時顯示 decision boundary、support vectors、margin、accuracy
5. 教學式解釋 C、kernel、gamma、degree、support vectors、margin 的意義
6. 提供 Kernel Trick 視覺化
7. 可以部署到 Streamlit Community Cloud

二、技術限制與設計原則

請嚴格遵守以下原則：

1. Streamlit app 不要即時呼叫 Manim。
2. Manim 動畫請放在 manim_scenes/ 資料夾。
3. Manim 渲染後的影片請放在 assets/videos/。
4. Streamlit 只使用 st.video() 播放影片。
5. sklearn SVM 的互動部分要在 Streamlit 中即時執行。
6. 資料量不要太大, 建議 n_samples 控制在 100 到 500 之間。
7. 使用 st.cache_data 或 st.cache_resource 提高效能。
8. 程式碼要模組化, 不要全部塞在 app.py。
9. 部署環境盡量簡單穩定, requirements.txt 不一定要包含 manim, 因為 Manim 只在本機生成影片。
10. 網站要適合教學, 不只是展示圖表, 每個互動參數都要有簡短解釋。

三、請建立以下專案結構

請建立類似以下結構：

```
svm-streamlit-teaching/
├── app.py
├── requirements.txt
├── README.md
└── .gitignore
```

```

assets/
├── videos/
│   ├── svm_margin_intro.mp4
│   ├── support_vectors_intro.mp4
│   └── kernel_trick_intro.mp4
└── images/

manim_scenes/
├── svm_margin_scene.py
├── support_vectors_scene.py
└── kernel_trick_scene.py

utils/
├── datasets.py
├── svm_model.py
├── plotting.py
└── explanations.py

pages/
├── 1_SVM_Concept.py
├── 2_Margin_and_Support_Vectors.py
├── 3_Interactive_SVM.py
├── 4_Kernel_Trick.py
└── 5_Quiz.py

```

如果你認為 Streamlit multi-page app 有更好的結構，也可以調整，但請保持清楚、可維護、可部署。

四、Streamlit 網站內容

請設計成多頁面教學網站，包含以下頁面：

Page 1：SVM 是什麼？

內容包含：

- SVM 的直覺介紹
- 二元分類問題
- 為什麼不是隨便找一條分隔線
- 最大 margin 的概念
- 播放 Manim 預先渲染影片：svm_margin_intro.mp4
- 用簡單圖示或文字解釋：
 - hyperplane
 - margin
 - support vector

Page 2：Margin 與 Support Vectors

內容包含：

- 決策函數：

$$f(x) = w \cdot x + b$$

- 分類規則：

$$\text{sign}(w \cdot x + b)$$

- margin 公式：

$$\text{margin} = 2 / ||w||$$

- SVM 最佳化問題：

$$\begin{aligned} &\text{minimize } 1/2 ||w||^2 \\ &\text{subject to } y_i(w \cdot x_i + b) \geq 1 \end{aligned}$$

- 解釋為什麼會最小化 $\frac{1}{2} ||w||^2$
- 播放 support_vectors_intro.mp4
- 用簡單中文解釋 support vectors 是真正影響邊界的資料點

Page 3 : Interactive SVM

這是核心互動頁面。

請在 sidebar 提供以下控制項：

1. Dataset 選擇：

- Linearly separable
- Blobs
- Moons
- Circles

2. Kernel 選擇：

- linear
- rbf
- poly

3. C slider：

- 範圍 0.01 到 100
- 建議使用 log scale 或 select slider
- 預設值 1.0

4. Gamma slider：

- 0.001 到 10
- kernel 為 rbf 或 poly 時顯示
- 預設值 scale 或 1.0

5. Degree slider：

- 2 到 5
- kernel 為 poly 時顯示

6. Noise slider：

- 0 到 0.5

7. n_samples slider：

- 100 到 500

請使用 sklearn 產生資料：

- make_classification
- make_blobs
- make_moons
- make_circles

請使用 sklearn.svm.SVC 訓練模型。

請畫出：

- 原始資料點
- decision boundary
- support vectors
- 如果 kernel 是 linear, 盡量顯示 margin lines
- training accuracy
- number of support vectors
- model parameters summary

請在圖旁邊加入教學解釋：

- C 越大代表模型越不容忍錯誤分類, 可能 overfit

- C 越小代表允許更多錯誤，margin 較寬，可能 underfit
- gamma 越大代表單一資料點影響範圍越小，邊界可能更彎曲
- gamma 越小代表邊界更平滑
- support vectors 是決定邊界的關鍵點

請讓圖表可以互動，優先使用 Plotly。

如果 Plotly 畫 decision boundary 太複雜，也可以使用 Matplotlib，但程式碼要清楚。

Page 4 : Kernel Trick

內容包含：

- Kernel Trick 的直覺解釋
- 為什麼 linear kernel 無法處理 circles / moons
- RBF kernel 如何讓非線性資料變得可分類
- 播放 kernel_trick_intro.mp4
- 提供互動比較：
 - linear kernel
 - rbf kernel
 - poly kernel

請使用 make_circles 或 make_moons 展示不同 kernel 的 decision boundary 差異。

請加上中文教學文字：

- linear kernel：適合線性可分資料
- rbf kernel：適合非線性、彎曲邊界
- poly kernel：使用多項式特徵空間，degree 越高邊界越複雜

Page 5 : Quiz

請建立簡單小測驗，讓使用者複習 SVM 概念。

至少包含 5 題選擇題：

1. Support vectors 是什麼？
2. C 變大通常會造成什麼效果？
3. gamma 變大通常會造成什麼效果？
4. margin 越大代表什麼？
5. RBF kernel 適合處理什麼資料？

請用 Streamlit radio button 實作，並在使用者作答後顯示答案與解釋。

五、Manim 動畫需求

請在 manim_scenes/ 裡面建立三個 Manim scene：

1. svm_margin_scene.py

動畫內容：

- 顯示兩群 2D 資料點
- 出現多條可分割直線
- 比較不同直線的 margin
- 最後保留最大 margin 的 decision boundary
- 標示 support vectors
- 顯示文字：「SVM chooses the hyperplane with the maximum margin」

2. support_vectors_scene.py

動畫內容：

- 顯示分類資料點
- 顯示 decision boundary
- 高亮離邊界最近的幾個點
- 標示它們為 Support Vectors
- 說明只有 support vectors 會直接決定 margin

3. kernel_trick_scene.py

動畫內容：

- 顯示 2D circles dataset
- 顯示線性分類器無法分開
- 用動畫表示資料被映射到更高維空間
- 在高維空間中用平面分開
- 顯示文字：「Kernel Trick helps SVM separate non-linear data」

請提供本機渲染指令，例如：

```
manim -pqh manim_scenes/svm_margin_scene.py SVMMarginScene
manim -pqh manim_scenes/support_vectors_scene.py SupportVectorsScene
manim -pqh manim_scenes/kernel_trick_scene.py KernelTrickScene
```

並且說明渲染後要把影片複製到：

assets/videos/

六、程式碼需求

請建立以下 utils：

utils/datasets.py

負責產生資料集。

需要有 function：

generate_dataset(dataset_name, n_samples, noise, random_state)

回傳 X, y。

utils/svm_model.py

負責訓練 SVM。

需要有 function：

train_svm(X, y, kernel, C, gamma, degree)

回傳 model, accuracy。

utils/plotting.py

負責繪圖。

需要有 function：

plot_decision_boundary(X, y, model)

需要顯示：

- data points
- decision regions
- decision boundary
- support vectors

如果是 linear kernel, 請盡量額外畫出 margin lines。

utils/explanations.py

放置教學文字。

例如：

```
explain_c_parameter()
explain_gamma_parameter()
explain_kernel()
explain_support_vectors()
```

七、部署需求

請建立 requirements.txt。

如果 Streamlit 部署端不需要 Manim, requirements.txt 請不要放 manim, 避免部署變複雜。

建議 requirements.txt：

```
streamlit
scikit-learn
numpy
pandas
matplotlib
plotly
```

如果你認為還需要其他套件，請加入並解釋原因。

請建立 README.md，內容包含：

1. 專案介紹
2. 本機執行方式
3. Manim 動畫渲染方式
4. Streamlit 本機啟動方式
5. Streamlit Community Cloud 部署方式
6. 常見問題排除

本機啟動：

```
streamlit run app.py
```

Manim 渲染：

```
manim -pqh manim_scenes/svm_margin_scene.py SVMMarginScene
```

部署到 Streamlit Community Cloud：

1. Push 專案到 GitHub
2. 到 streamlit.io 登入
3. 選擇 GitHub repo
4. main file path 設定為 app.py
5. Deploy

八、品質要求

請確保：

1. 程式碼可以直接執行
2. 沒有缺少 import
3. 檔案路徑正確
4. 沒有硬編碼絕對路徑
5. Streamlit app 不會因為找不到影片而 crash
6. 如果影片不存在，請顯示提示文字，例如：
「請先使用 Manim 渲染影片，並放到 assets/videos/」
7. 互動圖表在資料量 500 以下可以順暢運行
8. 所有教學文字使用繁體中文
9. 變數名稱和註解可以用英文，但教學網站內容請用繁體中文
10. README 要清楚到初學者可以照著部署

九、額外加分項目

如果時間允許，請額外加入：

1. SVM 參數比較表
2. 不同 kernel 的適用情境表格
3. Overfitting / Underfitting 提示
4. 使用者目前參數組合的解讀文字
5. 可以下載目前圖表的功能
6. 可以顯示 support vectors 數量占總資料比例
7. 小測驗分數統計
8. 頁面底部加入「下一步建議」

十、請你最後輸出

請你完成以下事情：

1. 建立完整專案檔案
2. 說明每個檔案用途
3. 給我本機執行指令
4. 給我 Manim 渲染指令
5. 給我 Streamlit 部署步驟
6. 檢查整個專案是否有潛在錯誤
7. 若有需要手動產生影片，請明確告訴我是哪幾個影片檔案
8. 請避免只給概念，請直接產生可用的程式碼

我會建議你丟給 Claude 之後，再補一句：

請先不要一次做太複雜，先完成 MVP 版本：

1. Streamlit 多頁面網站可以跑
2. sklearn SVM 互動 decision boundary 可以正常顯示
3. Manim scene 檔案先建立好
4. 影片不存在時用提示文字取代
5. 確認可以部署後，再美化 UI

這樣比較不會一開始就把專案做爆。

請依以下內容，生成一張清晰的「專案架構+工作流程」資訊圖表（infographic）。
風格：教學科技感、藍紅配色（對應 SVM 兩類資料）、分區塊、含流程箭頭。

專案名稱：SVM 互動式教學網站

一句話定位：用 Streamlit + scikit-learn + Plotly 打造的「支持向量機」互動教學網站，
 搭配 Manim 預渲染動畫，從直覺、數學到 Kernel Trick 完整教學。

線上 Demo：<https://l13-svm-interactivatpage.streamlit.app/>

GitHub：<https://github.com/ChenYuHsu413/L13-SVM>

【技術棧 Tech Stack】

- 前端/框架：Streamlit（多頁面 multi-page app）
- 機器學習：scikit-learn（SVC 模型、make_classification/blobs/moons/circles 資料集）
- 視覺化：Plotly（2D 決策邊界 + 3D 決策曲面，可互動旋轉）
- 概念動畫：Manim（僅本機離線渲染成 mp4）
- 語言：Python 3.12

【核心設計原則 Design Principle】

- ★ 關鍵：Streamlit 雲端「不」即時跑 Manim
 - Manim 只在本機把概念動畫渲染成 mp4
 - 網站只用 st.video() 播放預渲染影片
 - 互動式 SVM 視覺化才用 sklearn + Plotly 即時計算
 - 部署輕量穩定（requirements.txt 不含 manim）

【兩條工作流程 Workflow（重點，請用箭頭呈現）】

流程 A：離線動畫管線（本機，一次性）

- 撰寫 Manim Scene (.py)
 - 本機渲染 manim -qh ...（用獨立 Python 3.12 環境）
 - 輸出 mp4
 - 複製到 assets/videos/
 - commit 進 GitHub

流程 B：線上即時互動（使用者每次操作）

- 使用者在 sidebar 調參數（dataset / kernel / C / gamma / degree / noise / n_samples）
 - sklearn 產生資料 + 70/30 切分訓練/測試集
 - SVC 即時訓練（st.cache 加速）
 - Plotly 即時繪製：決策邊界、support vectors、margin、3D 曲面
 - 顯示訓練準確率 vs 測試準確率 + overfitting 提示

流程 C：部署管線 Deployment

GitHub repo → Streamlit Community Cloud → 設定 app.py → 自動安裝 requirements → 上線

【五個教學頁面 Pages】

1. SVM 是什麼？
 - 直覺介紹、二元分類、為什麼要最大 margin、hyperplane/margin/support vector 名詞
 - 播放動畫：svm_margin_intro.mp4
2. Margin 與 Support Vectors

- 決策函數 $f(x)=w \cdot x+b$ 、margin= $2/\|w\|$ 、最佳化問題 $\min \frac{1}{2}\|w\|^2$
- 互動：拉 C 即時看 margin 寬度變化
- 播放動畫：support_vectors_intro.mp4

3. Interactive SVM (核心互動頁) ★

- 即時調參數，2D 決策邊界 + 3D 決策曲面 (分頁切換)
- 顯示 support vectors、訓練 vs 測試準確率、overfitting/underfitting 提示
- 訓練點(●)/測試點(◆)疊圖、參數解讀、可下載 PNG

4. Kernel Trick

- 為什麼 linear 處理不了 circles/moons、RBF 如何升維分開
- 2D 邊界對比 + 3D 決策曲面對比 (linear/rbf/poly 並排)
- 播放 3D 動畫：kernel_trick_intro.mp4 (資料升維到高維被平面切開)

5. Quiz

- 5 題選擇題複習 + 即時對答案/解析 + 分數統計

【三支 Manim 概念動畫】

- svm_margin_intro：眾多分隔線中選出最大 margin
- support_vectors_intro：只有最近的點決定邊界
- kernel_trick_intro：2D 同心圓 → 升維到 3D → 平面切開 (3D 旋轉展示)

【專案結構 Structure】

app.py (首頁)
 pages/ (5 頁教學)
 utils/ (datasets / svm_model / plotting / explanations / media)
 manim_scenes/ (3 支動畫原始碼)
 assets/videos/ (預渲染 mp4)
 requirements.txt / README.md / .streamlit/config.toml

【教學亮點 Highlights】

- 每個互動參數都有繁體中文解釋
- 用真實 sklearn 模型畫真實邊界 (非示意圖)
- 訓練 vs 測試準確率讓 overfitting 「看得見」
- 2D + 3D 雙視角理解決策函數
- 影片不存在時顯示提示、不會 crash

請依據上述內容生成資訊圖表

顯示更多顯示較少

SVM 互動式教學網站

用 Streamlit + scikit-learn + Plotly 打造的「支持向量機」互動教學網站，
搭配 Manim 預渲染動畫，從直覺、數學到 Kernel Trick 完整教學。

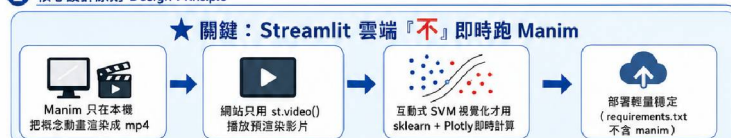
線上 Demo : <https://l13-svm-interactivepage.streamlit.app/>

GitHub : <https://github.com/ChenYuHsu413/L13-SVM>

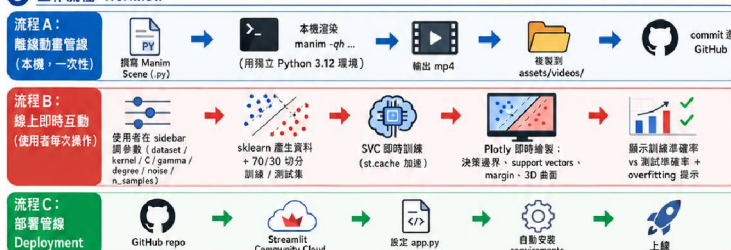
1 技術棧 Tech Stack



2 核心設計原則 Design Principle



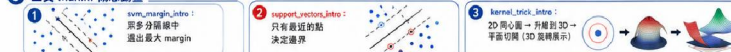
3 工作流程 Workflow



4 五個教學頁面 Pages



5 三支 Manim 概念動畫



6 專案結構 Structure

```

my.py (首頁)
pages/ (分頁)
utils/ (utils.py / cvx_model / plotting / explanations / media)
main_section/ (1 互動曲線圖)
assets/videos/ (預渲染 mp4)
requirements.txt
README.md
streamlit/config.toml

```

7 教學亮點 Highlights

